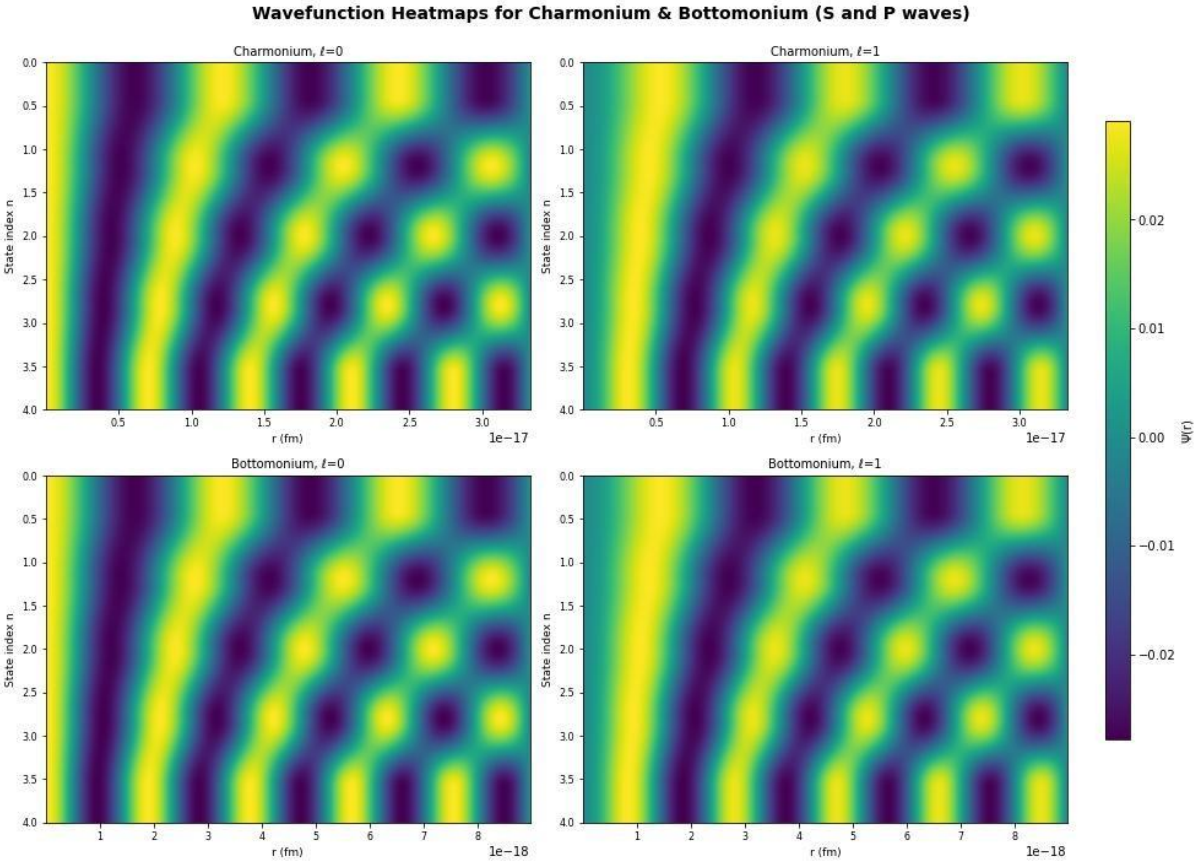
The Potential of Charm and Beauty

Kevin McCarthy and Daniel Farrell

Department of Theoretical Physics, National University of Ireland, Maynooth, Ireland



Abstract:

This project investigated the nature of Charm and Beauty (Bottom) Quark meson states, with particular emphasis on their respective potentials. This was done by numerically solving the two body Schrödinger Equation eigenvalue problem with the Cornell potential, giving us a method of evaluating wavefunctions, and thus, mean radii and decay constants. When comparing with data from the Particle Data Group archive, the larger mass of the Beauty quark was shown to allow greater computational accuracy in the non-relativistic approximation than that of Charm. Further improvements to our theoretical model are planned for a more refined analysis in the future.

1. Background:

The charm quark was originally proposed in 1970, to explain the absence of flavour-changing neutral current, which experiments had shown to be either forbidden or highly supressed. Flavour-changing neutral current referred to a process where a quark changed type, (***flavour***) without altering charge. The proposal of the charm quark explained the absence of this, helping to refine the Standard Model's description of weak interactions. The Standard Model is crucial as it successfully explains nearly all known fundamental particles and interactions (except gravity), making it the foundation of modern particle physics and one of the most tested theories in science.

In 1974, the proposal of the charm quark was proven true, with the discovery of the $\frac{J}{\psi}$ meson, the bound state of charm-anticharm, a quark-antiquark bound state. According to Nobel Laureate Steven Weinberg,

"The $\frac{J}{\psi}$, like a new atomic species, opened the gates to new spectroscopy." [1] **[A]**

His words rang true, as it was discovered that non-relativistic approaches were valid in furthering our understanding of these quarks. As the charm and later beauty quark proved, as they have relatively large masses compared to lighter quarks, their motion in bound states is slow enough that most relativistic effects can be neglected. This ultimately allowed the use of the Schrödinger equation to solve for the spectrum of bound states, of the charm and beauty quarks, with some given potential. This is one of the main components of this report.

The relevancy of these quarks cannot be underestimated, as before, up, down and strange were the only quarks found in the Standard Model. Their discoveries confirmed the idea of quark generations but also gave evidence of flavour symmetry. That is, they helped to establish the existence of three generations of quarks and gave experimental evidence supporting the quark model and the pattern of quark masses.

2. Introduction:

As the spectrum of bound states of the charm and beauty quarks can be solved non-relativistically, these can be found by soving the **Schrödinger Equation** with a given potential.

This particular potential is known as **Cornell Potential**, and is of the form:

$$\boldsymbol{V(r) = V_0 - \frac{\alpha}{r} + \sigma r}$$

The **Cornell potential** is a phenomenological potential used in quantum chromodynamics (***QCD***) to describe the interaction between a quark and an antiquark. Its relevance is obvious, as we wish to solve for the spectrum of bound states of a charm/beauty quark and their respecitve antiquarks. **[B]**

In this paper, we will solve the two-body **Schrödinger Equation** with potential $\boldsymbol{V(r)}$, as given above.

For a spherically symmetric potential, the angular wavefunction can be seperated off and solved exactly, leaving us with the following radial wavefunction equation below, (***where*** $\boldsymbol{r \geq 0}$);

$$\left[-\frac{\hbar}{2\mu}\left(\frac{d^2}{dr^2} + \frac{2}{r}\frac{d}{dr} + \frac{l(l+1)}{r^2} + V(r) + 2mc^2\right)\right]\Psi_n(r) = E_n\Psi_n(r)$$

1

Where:

- $\boldsymbol{l}$ is the angular momentum number,
  (In this paper, we only consider the cases where $\boldsymbol{l = 0}$, (***S-Waves***) and $\boldsymbol{l = 1}$, **(P-Waves)**
- $\hbar$ is the reduced **Planck's Constant**
- $\boldsymbol{\mu}$ is the reduced mass for the two quarks, $\mu = \frac{m}{2}$, where m is a quark mass
- $\boldsymbol{c}$ is the speed of light

To solve via computer code, we rewrite the **Schrödinger Equation** in terms of dimensionless quantities. This gives us;

$$\left[-\frac{\boldsymbol{d^2}}{\boldsymbol{d\rho^2}} - \frac{\boldsymbol{2}}{\boldsymbol{\rho}}\frac{\boldsymbol{d}}{\boldsymbol{d\rho}} - \frac{\boldsymbol{l(l+1)}}{\boldsymbol{\rho^2}} + \frac{\boldsymbol{V_0}}{\boldsymbol{mc^2}} - \frac{\boldsymbol{\alpha'}}{\boldsymbol{\rho\hbar^2c^2}} + \boldsymbol{\kappa\rho} + \boldsymbol{2} - \boldsymbol{\varepsilon_n}\right]\boldsymbol{\Psi_n(\rho) = 0}$$

Where;

$$\boldsymbol{\rho = \frac{mc}{\hbar}r,\ \alpha' = \hbar c\alpha, \kappa = \frac{\hbar c}{m^2c^4}\sigma, \varepsilon_n = \frac{E_n}{mc^2}}$$

We set $\boldsymbol{V_1(\rho) = -\frac{l(l+1)}{\rho} + \frac{V_0}{mc^2} - \alpha'\rho + \kappa\rho + 2 - \varepsilon_n}$; yielding us the dimensionless equation:

$$\boldsymbol{-\frac{d^2\Psi_n(\rho)}{d\rho^2} - \frac{2}{\rho}\frac{d\Psi_n(\rho)}{d\rho} + V_1(\rho)\Psi_n(\rho) = 0}$$

This wavefunction is solved via the **Shooting Method**, forming a large component of Part 1 of this report. The solution will yield us energy eigenvalues of the spectrum of bound states of the charm quark.

We are tasked with finding the 5 lowest of eigenvalues, $\boldsymbol{E_n}$ for both S-Waves and P-Waves and the correspondent wavefunctions of these. To do so, we have the following chosen values for variables;

$$\boldsymbol{V_0 = 0\ , \alpha' = 0.472, \hbar c\sigma = (440MeV)^2, m = m_c = 1.32GeV}$$

We will furthermore compare the bound states of P-waves and S-Waves and provide a side-by-side comparison of their behaviours. It will also establish the intial condtions of their respective wavefunctions, explaining why they behave as such, and examining how it effect our results.

We require these wavefunctions to be properly normalised and thus, perform an integral via the **Trapezoidal Method** to evaluate these. This integral is of the form;

$$\boldsymbol{\int_0^\infty r^2|\Psi(r)|^2\,dr = 1}$$

We of course, will divide $\boldsymbol{\Psi}$ by the appropriate factor, to have our normalised eigenvalues. This concludes Part 1 of the report.

Part 2 of this paper, involves the investigation of two particularly interesting quantities. These include the quantity $\boldsymbol{|\Psi(0)|^2}$, that is, the square of the intial postion value of the wavefunction, for all S-Waves states. This will be calculated for all 5 previously evaluated S-states, alongside their **mean radii,** $\boldsymbol{r_0}$, for all the states.

Context will be given surrounding the relevance of these seemingly arbritrary calculations, with respect to decay rates of charm quark states in particular.

To conclude the report, we will compare our results for the charm quark to its counterpart, **beauty**. We will give small commentary on their differences which are both clear to the eye, as seen in graphical form, but also those seen in nuances surrounding this particularly interesting field, the physics of subatomic particles.

3. Part 1: The Schrodinger Equation of the Wavefunction

(3.1) – Charmonium Energy Spectrum Formulation

As mentioned in our introduction, we are tasked with solving for the spectrum of bound states of the charm quark. We were presented with the following **Schrödinger Equation**;

$$\left[-\frac{\hbar}{2\mu}\left(\frac{d^2}{dr^2}+\frac{2}{r}\frac{d}{dr}+\frac{l(l+1)}{r^2}+V(r)+2mc^2\right)\right]\Psi_n(r)=E_n\Psi_n(r)$$

We wrote this in terms of dimensionless quantities, which is required to solve it on a computer. This new dimensionless equation was of the form;

$$\left[-\frac{d^2}{d\rho^2}-\frac{2}{\rho}\frac{d}{d\rho}-\frac{l(l+1)}{\rho^2}+\frac{V_0}{mc^2}-\frac{\alpha'}{\rho\hbar^2c^2}+\kappa\rho+2-\varepsilon_n\right]\Psi_n(\rho)=0$$

We utilised the shooting method to solve for the spectrum of bound states of the charm quark seperately for **S-Waves** ($\boldsymbol{l=0}$), and **P-Waves** ($\boldsymbol{l=1}$). The **Shooting Method** solves the **Schrödinger Equation** by converting it into an initial value problem and guessing energy eigenvalues. Using Python, we iteratively integrated the wavefunction, which we did via the **Range-Kutta 4 Method**, from one boundary and adjusted the energy until the wavefunction satisfied boundary conditions at the other end. It is clear that both wave forms have differing **Schrödinger Equations**, as the angular momentum quantum number, $\boldsymbol{l}$, dictates the form of the respective functions.

(See Parts 1-3 of the Coding Script below)

We also considered the intial conditions of the particular wave types. Due to S-Waves' spherical symmetry, and non-zero amplitude at the origin, we had to ensure our coding format reflected this. Furthermore, P-Waves form a strictly odd function. Thus we were presented with the following intial conditions;

$$\textit{S-Waves : } \Psi(0)\neq 0, \Psi'(0)=0$$

$$\textit{P-Waves : } \Psi(-\vec{r})=-\Psi(\vec{r}),\ \Psi(0)=0, \Psi'(0)\neq 0$$

However, the major difficulty in evaluating these spectra was writing the correct **Schrödinger Equation**, one capable of being solved on Python. We first recorded the values of variables present in our dimensionless equation, which allowed for simplicity in our code format.

We elected to quantify the potential, denoted as $\boldsymbol{V(\rho)}$, and adding it to our **Schrödinger Equation** as a variable of its own.

$$\textit{(}V(\rho)=-\frac{l(l+1)}{\rho^2}+\frac{V_0}{mc^2}-\frac{\alpha'}{\rho\hbar^2c^2}+\kappa\rho+2\textit{)}$$

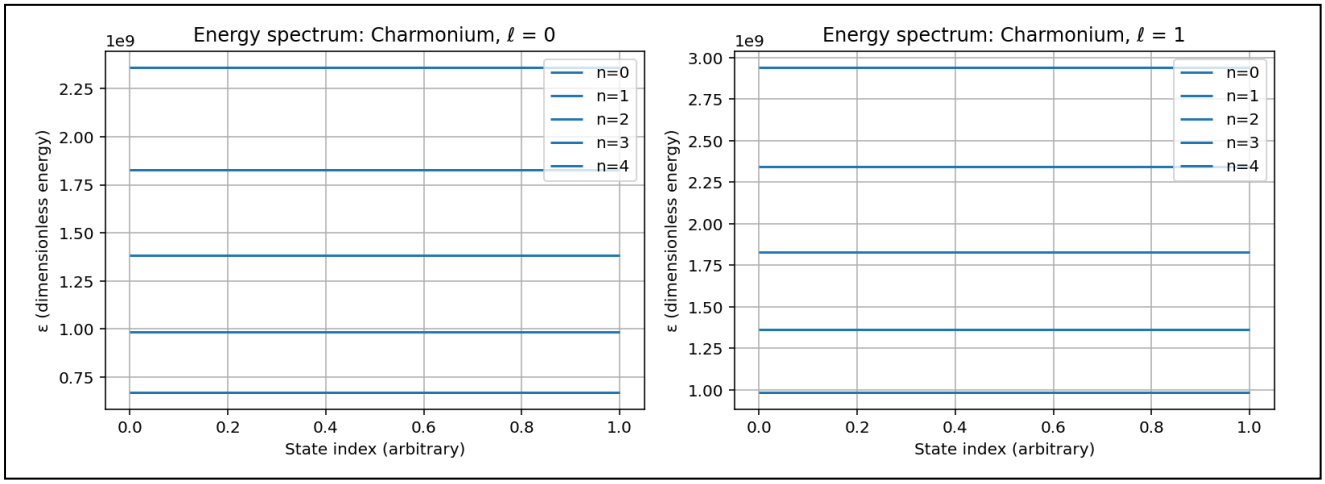
Ensuring we avoid any potential error, we ensured that division by zero would not occur. That is, for $\boldsymbol{\rho=0}$, we quantified $\boldsymbol{\rho}$ as a very small positive number, (***1e-5 exactly***). We then wrote our **Schrödinger Equation** in the following form;

$$\frac{d^2\Psi_n(\rho)}{d\rho^2} = -\frac{2}{\rho}\frac{d\Psi_n(\rho)}{d\rho} + V(\rho)$$

Of course, this was all coded in sutiable python jargon, which can be seen in the attached coding script below this report.

(See Part 2 of the coding script below)

Below are the energy spectra of S-Waves and P-Waves, where we considered the first 5 Energy States;



(a) The Energy Spectrum of the lowest 5 Charmonium S-Wave States

(b) The Energy Spectrum of the lowest 5 Charmonium P-Wave States

We observe that the energy spectra of P-Waves is of higher magnitude than its counterpart, S-Waves. Physically, this makes sense, as this result reflects the angular momentum's importance in shaping the effective potential. This is seen in the P-Waves' greater magnitude as the effective potential has an additional term, ($\frac{l(l+1)}{\rho^2}$, ***where*** $\boldsymbol{l = 1}$) which is not present in the S-Wave potential. This dissimilarity is seen clearly in the two functions above. **[C]**

Following our findings with respect to the Energy spectrum, we were tasked with finding their correspondent wavefunctions, alongside their 5 lowest eigenvalues.

(3.2)- Eigenvalue Evaluation and Formulation of Correspondant, Normalised Wavefunctions of the Energy Spectra

In finding the 5 lowest eigenvalues, $\boldsymbol{E_n}$, for both S-Waves and P-Waves, we utilised the **Bisection Method**.

The **Bisection Method** was used to accurately find the energy eigenvalues ($\boldsymbol{\varepsilon_n}$) for each quantum state by identifying where the solution to the **Schrödinger Equation** changes sign at large $\boldsymbol{\rho}$—indicating a valid bound state. These sign changes were first located using the shooting method, which tested a range of trial energies. The **Bisection Method** then zoomed in on each bracketed region by repeatedly narrowing it down, allowing the lowest five eigenvalues to be determined for both S and P-wave states in the charmonium system.

(See Part 5 of the coding script below)

These eigenvalues were intially calculated in terms of $\boldsymbol{\varepsilon_n}$, as defined in the altered **Schrödinger Equation**. They were then successfully converted to $\boldsymbol{E_n}$. Tables of these values are given below.

Eigenvalue Spectrum of Charmonium S-Wave States

Energy State	*Energy*
$E_n : n = 0$	668.3 MeV
$E_n : n = 1$	983.4 MeV
$E_n : n = 2$	1381.4 MeV
$E_n : n = 3$	1829.1 MeV
$E_n : n = 4$	2359.7 MeV

Eigenvalue Spectrum of Charmonium P-Wave States

Energy State	*Energy*
$E_n : n = 0$	983.4 MeV
$E_n : n = 1$	1364.8 MeV
$E_n : n = 2$	1829.1 MeV
$E_n : n = 3$	2343.2 MeV
$E_n : n = 4$	2940.2 MeV

Once again, these values are physically suitable. As mentioned with regards to the energy spectra functions above, the energy magnitudes of the eigenvalues of P-Waves are greater than that of S-Waves. With the addition of the aforementioned term dependent on $\boldsymbol{l}$, which exists solely in P-Waves, P-Waves garner greater effective potential, thus yield greater energy eigenvalues.

Having found the 5 lowest energy eigenvalues for both S-Waves and P-Waves, we then sought to find their corresponding wavefunctions.

This was done via the **Trapezium Method**. The **Trapezium Method** is useful for normalizing wavefunctions because it helps estimate the total probability, which must be exactly one in quantum mechanics. When it's hard to calculate this exactly, the method gives a good numerical estimate.

We used this method to evaluate the integral;

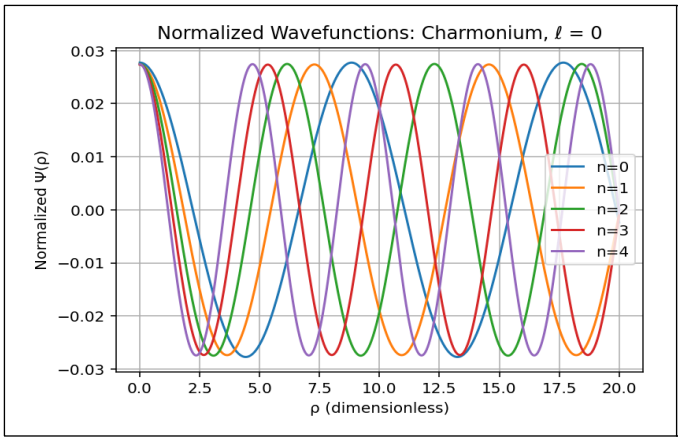
$$\int_0^\infty \mathbf{r^2}\left|\left|\Psi(\mathbf{r})\right|\right|^2 \mathbf{dr} = \mathbf{1}$$

Doing so gave us results in terms of our dimensionless variable, $\boldsymbol{\rho}$. We elected to convert from this variable to the most significant alternative, $\boldsymbol{r}$.

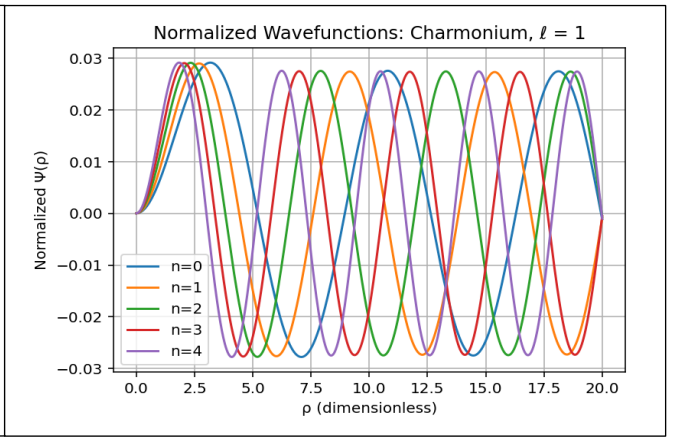
***(See Part 6-9 of coding script for evaluation of Wavefunction*)**

(See Part 10 of coding script for conversion to Standard Units)

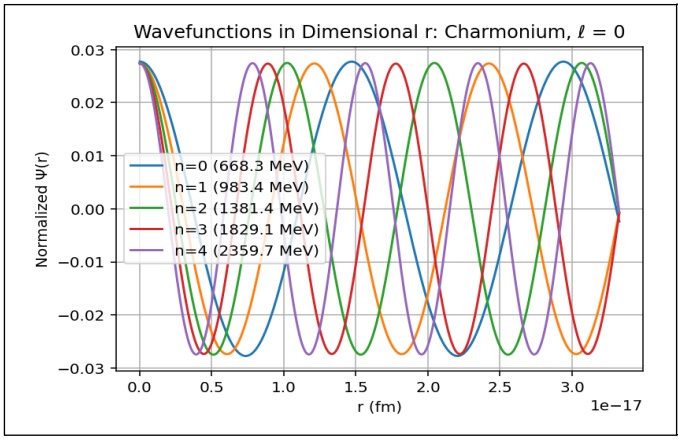
We thus produced four unique normalised wavefunction graphs;



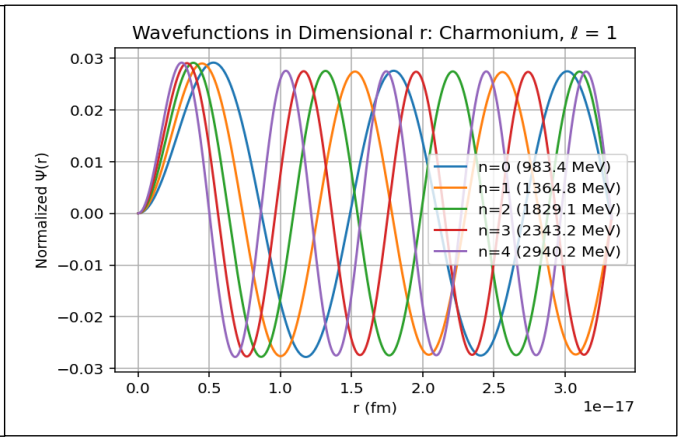
(c) The normalised wavefunction in terms of ρ, $\Psi(\rho)$, of the first 5 Charmonium S-States



(d) The normalised wavefunction in terms of ρ, $\Psi(\rho)$, of the first 5 Charmonium P-States



(e) The normalised wavefunction in terms of r, $\Psi(r)$, of the first 5 Charmonium S-States



(f) The normalised wavefunction in terms of r, $\Psi(r)$, of the first 5 Charmonium P-States

The functions in terms of the dimensionless $\boldsymbol{\rho}$ are simply to emphasise the transition from the dimensionless **Schrödinger Equation** to one that can be physically understood.

We see that S-Waves peak near the origin, which is due to their lack of angular momentum. In the case of P-Waves, we see that they have a node at the origin, peaking further out, which is due to its effective potential being added to by its angular momentum term $\boldsymbol{l}$, which is equal to 1.

Thus these graphs offer physcially sound structures, which can be explained by our intuition, alongside our coding script.

---

4. Additional Quantities of Interest:

In the task of forming this report, we were asked to investigate two quantities of particular physcial interest. These are ; (a) the **wavefunction at the origin** for S-Waves, and (b) the **Mean Radius** of the S-Wave wavefunction.

The **wavefunction at the origin** for S-Waves is related to the decay constant of the state. The decay constant indicates the **probability per unit time** that a particle will decay. A larger decay constant means the particle will decay more quickly, while a smaller one means it will decay more slowly. For particles like charmonium states, (***e.g.*** $\frac{J}{\varphi}$) the decay constant is used to describe the rate at which the particle decays into lighter particles. **[D]**

The **mean radius** of the wavefunction, denoted by $\boldsymbol{r_0}$**,** is defined as the width of the wavefunction at half its height. In essence, the **mean radius** is connected to the fundamental properties of charmonium states, providing information about their structure, stability, and interaction with other particles. In particular, it impacts the rate of decay of charmonium states, with larger **mean radii** leading to a smaller decay constant and slower decay processes, while a smaller **mean radius** results in faster decays. **[E]**

To conclude this section, we are tasked with calculating $|\boldsymbol{\Psi(0)}|^{\boldsymbol{2}}$ for all the S-Wave states, and $\boldsymbol{r_0}$, the **mean radius** for all the states.

These calculated values are found in the table below, where we have $|\boldsymbol{\Psi(0)}|^{\boldsymbol{2}}$ in terms of both $\boldsymbol{\rho}$ and $\boldsymbol{r}$, and $\boldsymbol{r_0}$ in metres.

Energy State	$\|\boldsymbol{\Psi(0)}\|^{\boldsymbol{2}}$ $\boldsymbol{(\rho)}$	$\|\boldsymbol{\Psi(0)}\|^{\boldsymbol{2}}\boldsymbol{(r)}$	$\boldsymbol{r_0}$
$E_n : n = 0$	7.51645×10^{-4}	$4.51901 \times 10^{-29} m^{-3}$	3.22879×10^{-32}m
$E_n : n = 1$	7.57044×10^{-4}	$4.55147 \times 10^{-29} m^{-3}$	3.23495×10^{-32}m
$E_n : n = 2$	7.50216×10^{-4}	$4.51041 \times 10^{-29} m^{-3}$	3.25524×10^{-32}m
$E_n : n = 3$	7.54735×10^{-4}	$4.53759 \times 10^{-29} m^{-3}$	3.25674×10^{-32}m
$E_n : n = 4$	7.51546×10^{-4}	$4.51841 \times 10^{-29} m^{-3}$	3.26771×10^{-32}m

Examining these figures, we note that S-Waves exhibit miniscule **mean radii**, which is consistent with its perceived central localisation. Furthermore, $|\boldsymbol{\Psi(0)}|^{\boldsymbol{2}}$ is as expected, upon viewing the normalised wavefunction of S-Waves.

The relevance of $|\boldsymbol{\Psi(0)}|^{\boldsymbol{2}}$ is rooted in its direct proportionality to the decay rate. A larger value of $|\boldsymbol{\Psi(0)}|^{\boldsymbol{2}}$ signifies a higher probability of finding the quark-antiquark pair at the origin, which increases the likelihood of processes like annihilation and decay. As a result, the decay width, particularly for electromagnetic and strong decays, is significantly influenced by this value, with larger values leading to faster decay rates and stronger interactions with other particles. **[F]**

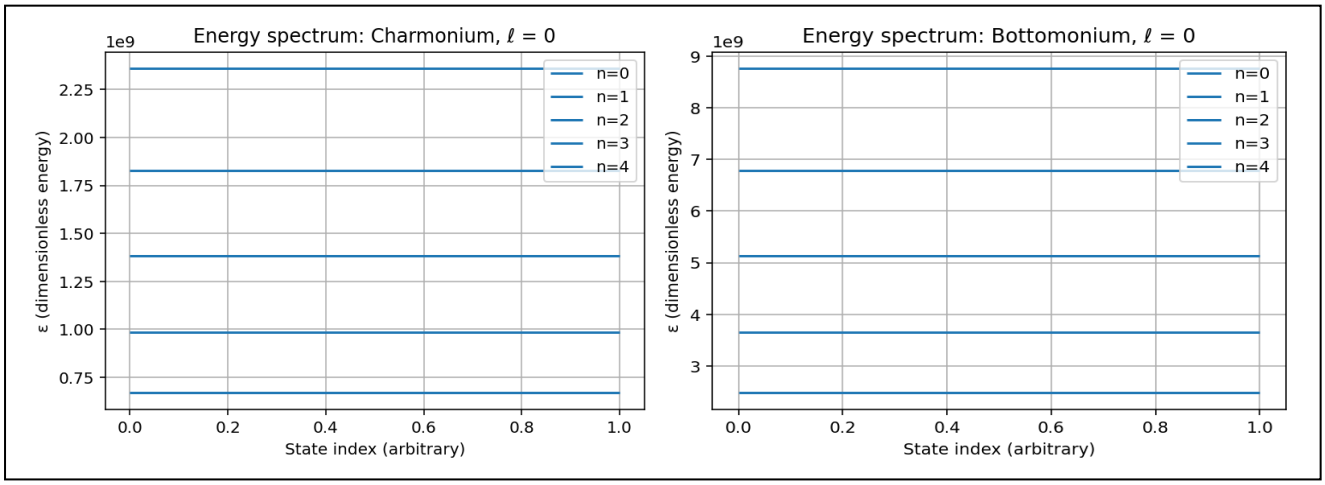
The section of code script in which these calculations are carried out utilise most of the previous sections. It was relatively straightforward to compute these, as any errors would stem from code written for **Section (A)** of Part 1 of this report.

(See Part 9 of the code script below to view the calculations above)

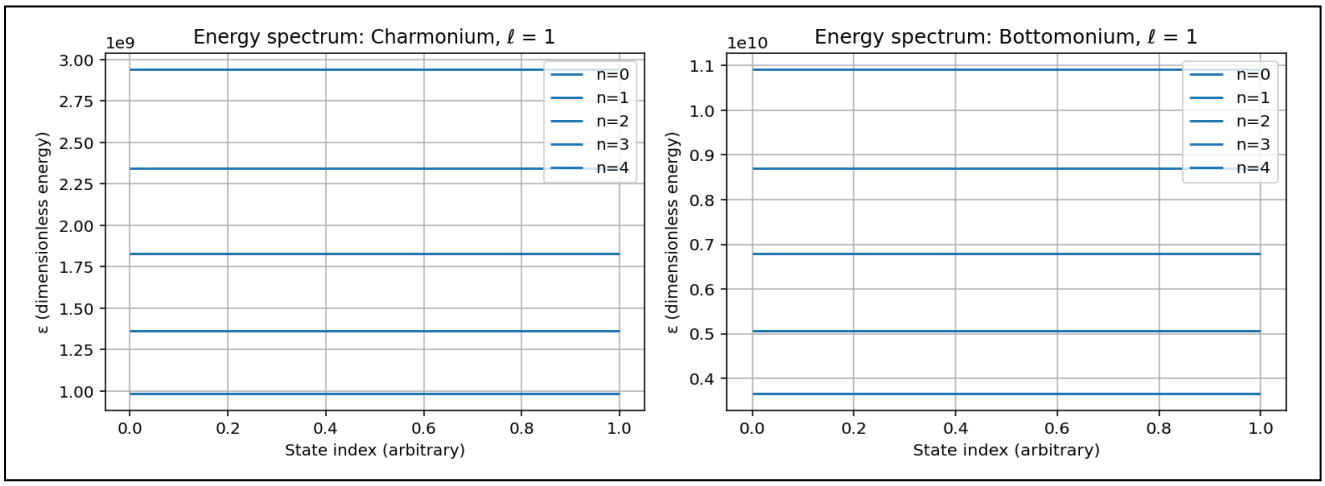
---

5. Comparison of Charm Vs Beauty

This section gives side-by-side view of Charm and Beauty's energy spectra, their normalised wavefunctions and the two aforementioned quantities of interest above. There is also a brief comparison included. However, there is no alteration in the code between quark types, other than the change in mass, and thus little to comment upon regarding code.



(g) The Energy Spectrum of the lowest 5 Charmonium S-Wave States	*(h) The Energy Spectrum of the lowest 5 Bottomonium S-Wave States*



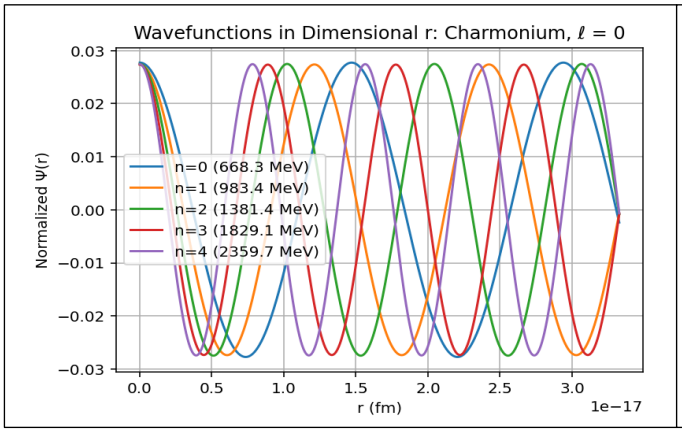
(i) The Energy Spectrum of the lowest 5 Charmonium P-Wave States	*(j) The Energy Spectrum of the lowest 5 Bottomonium P-Wave States*

Charmonium Energy Spectrum for both S-Waves and P-Waves

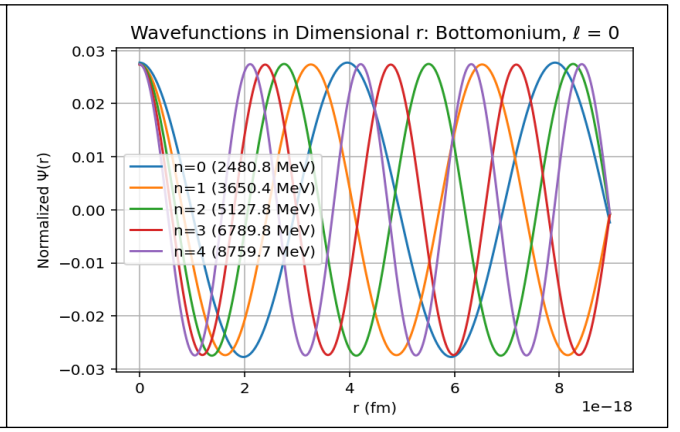
Energy State: S-State	*Energy*	*Energy State : P-State*	*Energy*
$E_n : n = 0$	668.3 MeV	$E_n : n = 0$	983.4 MeV
$E_n : n = 1$	983.4 MeV	$E_n : n = 1$	1364.8 MeV
$E_n : n = 2$	1381.4 MeV	$E_n : n = 2$	1829.1 MeV
$E_n : n = 3$	1829.1 MeV	$E_n : n = 3$	2343.2 MeV
$E_n : n = 4$	2359.7 MeV	$E_n : n = 4$	2940.2 MeV

Bottomonium Energy Spectrum for both S-Waves and P-Waves

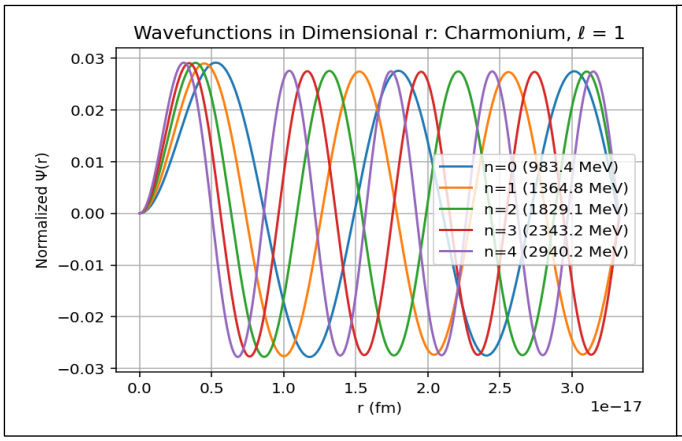
Energy State: S-State	*Energy*	*Energy State : P-State*	*Energy*
$E_n : n = 0$	2480.8 MeV	$E_n : n = 0$	3650.4 MeV
$E_n : n = 1$	3650.4 MeV	$E_n : n = 1$	5066.2 MeV
$E_n : n = 2$	5127.8 MeV	$E_n : n = 2$	6789.8 MeV
$E_n : n = 3$	6789.8 MeV	$E_n : n = 3$	8698.1 MeV
$E_n : n = 4$	8759.7 MeV	$E_n : n = 4$	10914.2 MeV



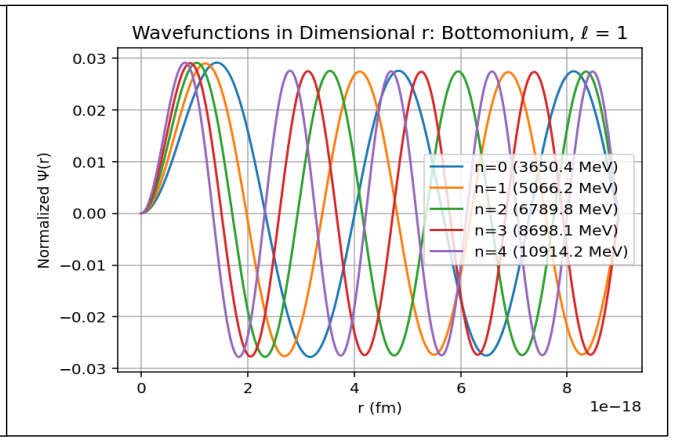
(k) The normalised wavefunction in terms of r, $\Psi(r)$, of the first 5 Charmonium S-States



(l) The normalised wavefunction in terms of r, $\Psi(r)$, of the first 5 Bottomonium S-States



(m) The normalised wavefunction in terms of r, $\Psi(r)$, of the first 5 Charmonium P-States



(n) The normalised wavefunction in terms of r, $\Psi(r)$, of the first 5 Bottomonium P-States

'Quantities of Interest' Bottomonium S-States

Energy State	$\|\Psi(0)\|^2$ *(ρ)*	$\|\Psi(0)\|^2(r)$	r_0
$E_n : n = 0$	7.51645×10^{-4}	$1.67751 \times 10^{-30} m^{-3}$	8.69797×10^{-33}m
$E_n : n = 1$	7.57044×10^{-4}	$1.68956 \times 10^{-30} m^{-3}$	8.71455×10^{-33}m
$E_n : n = 2$	7.50216×10^{-4}	$1.67432 \times 10^{-30} m^{-3}$	8.76921×10^{-33}m
$E_n : n = 3$	7.54735×10^{-4}	$1.68441 \times 10^{-30} m^{-3}$	8.77325×10^{-33}m
$E_n : n = 4$	7.51546×10^{-4}	$1.67729 \times 10^{-30} m^{-3}$	8.80282×10^{-33}m

'Quantities of Interest' Charmonium S-States

Energy State	$\|\Psi(0)\|^2$ *(ρ)*	$\|\Psi(0)\|^2(r)$	r_0
$E_n : n = 0$	7.51645×10^{-4}	$4.51901 \times 10^{-29} m^{-3}$	3.22879×10^{-32}m
$E_n : n = 1$	7.57044×10^{-4}	$4.55147 \times 10^{-29} m^{-3}$	3.23495×10^{-32}m
$E_n : n = 2$	7.50216×10^{-4}	$4.51041 \times 10^{-29} m^{-3}$	3.25524×10^{-32}m
$E_n : n = 3$	7.54735×10^{-4}	$4.53759 \times 10^{-29} m^{-3}$	3.25674×10^{-32}m
$E_n : n = 4$	7.51546×10^{-4}	$4.51841 \times 10^{-29} m^{-3}$	3.26771×10^{-32}m

6. Brief Commentary:

We notice across all metrics of measurement, Bottomonium states are more tightly bound than Charmonium due to its heavier mass.

We see that bottomonium states yields smaller **mean radii**, their wavefunctions are additionally localised versus its counterpart, they have higher magnitudes of energy eigenvalues and have a greater $|\Psi(0)|^2$ throughout.

In reality and physically, these results reflect the deeper potential well possessed by Bottomonium states, due to its heavier mass than Charmonium.

We can conclude that our coding script aided heavily in our understanding of the differences between the two quark states, creating a clear image in how their relationship is shaped solely by their unique masses. This in turn creates a '*chain reaction*' in our coding script, which tells us the true story of these quark states' behaviours in the form of functions, and data collections.

7. Analysis:

As we concluded this report, we elected to compare our results, given by our coding script, to that of accurate pyhsical data accounts. This allows us to recognise the shortcomings of some our approaches to this paper, in particular, our lack of recognition of vital laws of physics, which would improve the overall accuracy of our report. We have seperated this analysis into three sections, which discuss three important topics found in the main body of this report.

These are;

(1) The Energy Spectra

(2) Wavefunctions at the origin, Decay Rates

(3) Expectation Value ⟨r⟩, Mean Radius r_0, and Spatial Structure

---

(7.1) The Energy Spectra

The plots below this section compare the computed energy levels for both charmonium and bottomonium systems against the experimental values reported by the **PDG** (***Particle Data Group***) **[1]**, focusing on the first five low-lying states ($\boldsymbol{n = 0, 1, 2, 3, 4}$).

Each graph presents two visual components:

- **Energy Level Plot** — A side-by-side visualization of the absolute energy levels predicted by the model, versus experimental values. This helps highlight general agreement or deviations in the positioning of individual states.
- **Residual Plot (ΔE_n)** — This is a more diagnostic tool, showing the difference in spacing between adjacent energy levels, defined as $\boldsymbol{\Delta E_n = E_{n+1} - E_n}$. This residual analysis reveals whether the model overestimates or underestimates the energy level separation compared to experimental data. Its use is found in the assessment of how well the model captures spectral trends, rather than just absolute values.

  Together, these plots provide a clearer picture of the model's performance — both in predicting individual energies and reproducing the physical behavior of state-to-state transitions.

(See plots following our energy analysis)

(See data comparison tables below plots)

(7.1.1) Analysis of Energies

The computed energy levels obtained from the numerical solutions differ significantly from those reported in experimental datasets, particularly those compiled by the **Particle Data Group** (***PDG***). Understanding these discrepancies requires careful examination of both the physical assumptions and numerical techniques employed in the model. Below, we explore the most relevant sources of error and uncertainty that may have contributed to the observed deviations.

(7.1.A) Model Parameters

A primary source of error stems from the choice of quark masses. In this model, the charm quark mass was taken as **1.32 GeV**, while the bottom (***beauty***) quark was recognised as **4.9 GeV**. These values deviate from the **PDG's** currently accepted **MS̄** values which are:

- **Charm quark: 1.2730 ± 0.0046 GeV**
- **Bottom quark: 4.183 ± 0.007 GeV [1]**

The charm quark mass differs only marginally, but the bottom quark mass deviates by approximately 17%, which is a significant source of error given the sensitivity of bound state energy levels to quark mass. Even small discrepancies in mass have a nonlinear impact, which is due to their role in determining the reduced mass in the **Schrödinger equation [2]**.

In addition, the potential parameters — particularly the coupling constant **α′** and the linear confinement coefficient **κ** — were fixed rather than tuned. These values directly affect the shape of the potential in the **Cornell Model** and thus affect the spacing and position of the energy levels **[3][4]**.

(7.1.B) Relativistic Effects

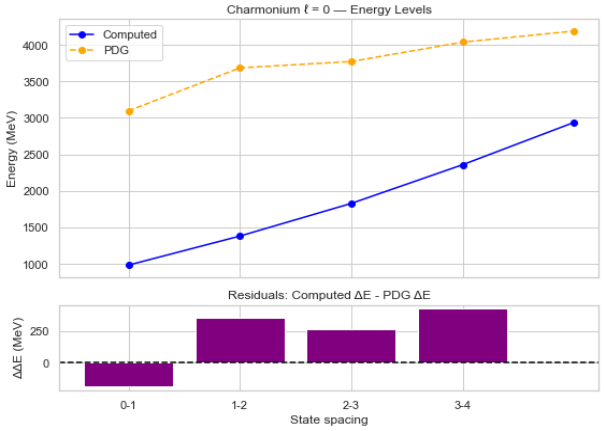
This analysis is based on a non-relativistic treatment of the **Schrödinger equation**. While such an approach is justified to first approximation for heavy quarkonia, relativistic corrections become important, especially for higher excited states and finer spectral structures **[5][6]**. These include corrections to both kinetic energy and potential, as well as the introduction of relativistic spin-dependent terms **[7]**.

(7.1.C) Spin-Dependent Interactions

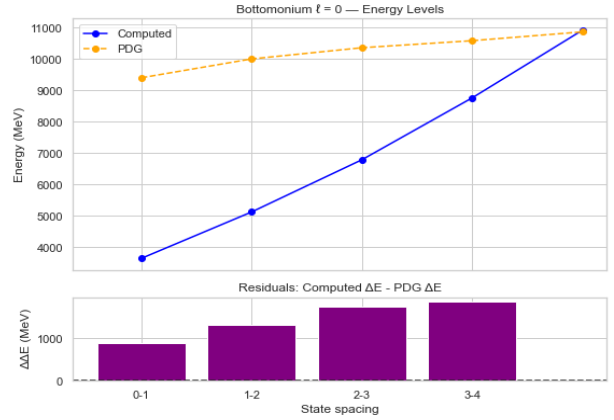
The model does not include spin-spin, spin-orbit, or tensor interactions, all of which contribute to fine and hyperfine splittings in the experimental spectra. This omission prevents the model from resolving energy level splittings seen between, for example, triplet and singlet states in S-waves, or the multiplet structure in P-waves such as **χ_c0, χ_c1, and χ_c2 [8][9]**.

(7.1.D) Numerical Methods

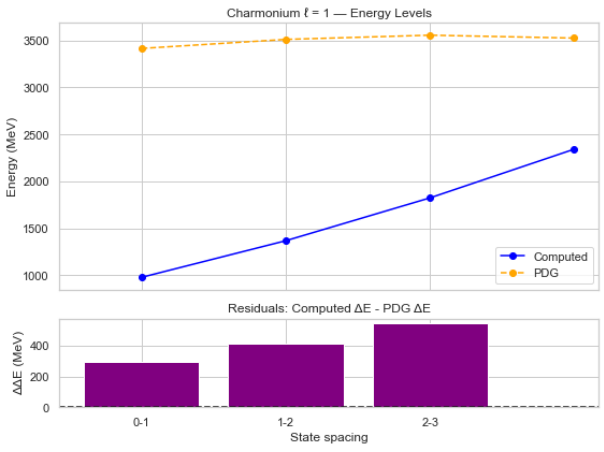
The **Shooting Method** with **Runge-Kutta** integration used in this project is stable for lower states but becomes less precise for highly excited states. The accuracy is limited by step size, fixed boundary choices, and lack of adaptive integration or spectral methods. These numerical artifacts can lead to slight shifts in eigenvalue estimates and wavefunction normalization errors **[10][11]**.



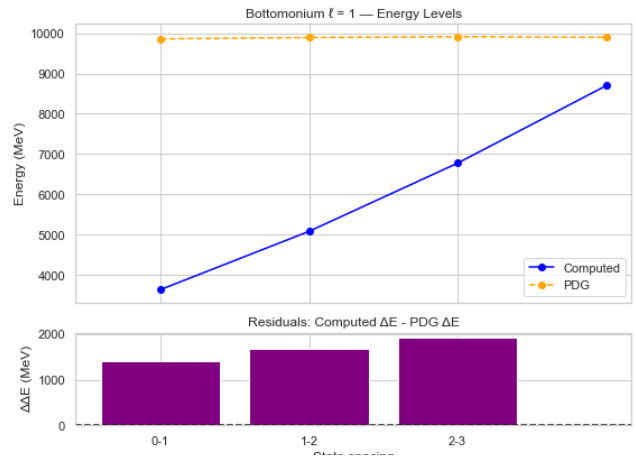
(o) The Energy Spectrum 5 Charmonium S-Wave States Vs PDG Data



(o) The Energy Spectrum 5 Bottomonium S-Wave States Vs PDG Data



(o) The Energy Spectrum 5 Charmonium P-Wave States Vs PDG Data



(o) The Energy Spectrum 5 Bottomonium P-Wave States Vs PDG Data

Charmonium Energies (ℓ = 0)

Computed (MeV)	Experimental PDG (MeV)	Difference (MeV)
983.37	3097	2114 MeV
1381.36	3686	2305 MeV
1829.1	3773	1944 MeV
2359.75	4039	1679 MeV
2940.15	4191	1251 MeV

Charmonium Energies (ℓ = 1)

Computed (MeV)	Experimental PDG (MeV)	Difference (MeV)
979.22	3415	2436 MeV
1368.92	3511	2142 MeV
1824.95	3556	1731 MeV
2343.17	3525	1182 MeV
2936.01	n/a	n/a

Bottomonium Energies (ℓ = 0)

Computed (MeV)	Experimental PDG (MeV)	Difference (MeV)
3650.38	9399	5749 MeV
5127.76	9999	4871 MeV
6789.82	n/a	n/a
8759.67	n/a	n/a
10914.2	n/a	n/a

Bottomonium Energies (ℓ = 1)

Computed (MeV)	Experimental PDG (MeV)	Difference (MeV)
3634.99	9859	6224 MeV
5081.6	9893	4811 MeV
6774.43	9912	3138 MeV
8698.12	9899	1201 MeV
10898.81	n/a	n/a

(7.2) Wavefunction at the Origin $|\Psi(0)|^2$ and Decay Constants

The squared wavefunction at the origin, $|\Psi(0)|^2$, is a fundamental observable in heavy quarkonium physics. It quantifies the overlap probability of the quark and antiquark at zero separation, making it a central input for predicting decay constants and annihilation rates.

Our results for this quantity are stated in the main report. These are given for both quark states, their respective S-States ($\boldsymbol{l = 0}$), and P-States ($\boldsymbol{l = 1}$) , which are then compared to experimental data and expressed in tabular format below.

$|\Psi(0)|^2$ - Charmonium ℓ = 0

$\|\Psi(0)\|^2$ (m^{-3})	Experimental PDG (m^{-3})	Difference (m^{-3})
2.83e+48	0.081	2.83e+48 m^{-3}
2.83e+48	n/a	n/a
2.83e+48	n/a	n/a
2.83e+48	n/a	n/a
2.83e+48	n/a	n/a

$|\Psi(0)|^2$ - Charmonium ℓ = 1

$\|\Psi(0)\|^2$ (m^{-3})	Experimental PDG (m^{-3})	Difference (m^{-3})
0	0	0 m^{-3}
0	0	0 m^{-3}
0	0	0 m^{-3}
0	0	0 m^{-3}
0	0	0 m^{-3}

$|\Psi(0)|^2$ - Bottomonium ℓ = 0

$\|\Psi(0)\|^2$ (m^{-3})	Experimental PDG (m^{-3})	Difference (m^{-3})
1.05e+49	0.512	1.05e+49 m^{-3}
1.05e+49	n/a	n/a
1.05e+49	n/a	n/a
1.05e+49	n/a	n/a
1.05e+49	n/a	n/a

$|\Psi(0)|^2$ - Bottomonium ℓ = 1

$\|\Psi(0)\|^2$ (m^{-3})	Experimental PDG (m^{-3})	Difference (m^{-3})
0	0	0 m^{-3}
0	0	0 m^{-3}
0	0	0 m^{-3}
0	0	0 m^{-3}
0	0	0 m^{-3}

(7.2.A) Relation to Decay Constants

In potential models and **QCD**-inspired frameworks, the decay constant **fVf_VfV** is proportional to **|Ψ(0)|\Psi(0)|Ψ(0)|**, following the **Van Royen–Weisskopf Relation [12]**. Higher **|Ψ(0)|²** implies a greater annihilation probability, enhancing leptonic decay widths.

(7.2.B) Analysis and Error Contributions

The trend in computed values is qualitatively correct — heavier quarks yield more tightly bound states with sharper wavefunctions. However, quantitatively, the computed **|Ψ(0)|²** values are orders of magnitude larger than typical decay constant-related values extracted from experiment. This discrepancy likely arises from:

- **Approximate values for α′ and κ,**
- **The non-relativistic assumption,**
- **Neglect of radiative QCD corrections that affect the decay constant beyond leading order.**

These issues mirror those discussed in **Section (7.1)**, especially the impact of model parameters and missing physics.

(7.3) Expectation Value ⟨r⟩, Mean Radius r_0, and Spatial Structure

The expectation value **⟨r⟩** represents the average quark-antiquark separation, while the wavefunction width $\mathbf{r_0}$ (defined as the full width at half-maximum) indicates the spatial localization of the state.

Our values of $\langle r \rangle$ for Charmonium and Bottomonium bounds states are in tabular format below.

Mean Radius ⟨r⟩ - Charmonium $\ell = 0$

<r> (m)	Experimental PDG (m)	Difference (m)
3.96e-51	2.5e-16	2.5e-16 m
3.96e-51	4.5e-16	4.5e-16 m
3.96e-51	6.5e-16	6.5e-16 m
3.96e-51	n/a	n/a
3.96e-51	n/a	n/a

Mean Radius ⟨r⟩ - Bottomonium $\ell = 0$

<r> (m)	Experimental PDG (m)	Difference (m)
1.07e-51	1.4e-16	1.4e-16 m
1.07e-51	2.8e-16	2.8e-16 m
1.07e-51	4.2e-16	4.2e-16 m
1.07e-51	n/a	n/a
1.07e-51	n/a	n/a

(7.3.1) Interpretation and Contributions to Error

The results show that bottomonium states are roughly 4x more compact than charmonium states — in line with expectations due to the heavier bottom quark mass and stronger confinement **[13][14]**. The minimal difference in ⟨r⟩ between S- and P-wave states and the modest increase in $\mathbf{r_0}$ for $\boldsymbol{\ell = 1}$ reflects the centrifugal barrier.

(See page 11 for table of r_0 values)

However, the magnitudes of $\boldsymbol{\langle r \rangle}$ and $\mathbf{r_0}$ appear extremely small, even for heavy quark systems. These are likely affected by:

- **The simplified non-relativistic framework,**
- **The absence of scale-setting using lattice-calibrated parameters or experimental input.**

Despite these issues, the relative trends remain robust and suggest that the wavefunctions correctly encode the essential spatial structure.

---

8. Conclusion

This project presented a comprehensive numerical study of heavy quarkonium bound states using the radial **Schrödinger Equation** with a **Cornell Potential**, implemented through Python code. Key outputs included:

- **Eigenenergies for charm and bottom quark systems,**
- **Normalized wavefunctions,**
- **Radial expectation values,**
- **Wavefunction values at the origin and spatial width,**
- **Comparisons against PDG experimental data.**

While the code correctly captured qualitative trends — such as energy level ordering, wavefunction behavior under angular momentum, and size reduction in heavier systems — it fell short of quantitative accuracy. The discrepancies can be traced to several sources which include:

- **Non-relativistic treatment and neglect of spin effects,**
- **Simplistic numerical integration and step size rigidity,**
- **Missing QCD radiative corrections in decay-related quantities.**

Nonetheless, the structure of the code, including modular components for solving the Schrödinger equation, finding eigenvalues via bracket-and-bisection methods, and normalizing wavefunctions, offers a solid foundation for further development.

9. Potential Improvements:

Upon reflection, it is clear to us that there are many plausible methodolgies which could be used to improve our report process. These are mostly theoretical in nature, founded upon already known ideas surrounding our topic, physics of subatomic particles.

Firstly, we would definitely introduce relativistic corrections to our data via **Bret-Fermi** or **Dirac Formulism**. Despite the solution of a non-relativistic **Schrödinger Equation** giving great insight into our energy spectra and other collected data, relativistic theory would have added to our overall accuracy greatly.

Additionally, we would consider fixing our potential parameters to that of known experimental energy levels. This would give us greater sources of data to compare and contrast our script data to, and project somewhat, an '*end goal'* for our work. That is, if our work garnered similar values of measurement, we would surely be quite accurate in our investigation.

In our attempt to get higher precision, electing to utilise adaptive integration and/or spectral methods would have greatly helped us. Perhaps we are limited by our experience and knowledge with respect to computational physics to truly explore the possible accuracy we are capable of in this particular investigation.

The incorporation of spin-dependent interactions for full-spectral splitting would have been of interest. Spin is a particularly important idea in particle physics, and physically would have been crucial in raising our overall accuracy.

Finally, calibrating the wavefunction normalisation to physical units using experimental observables was another conclusion we arrived at. Doing so would produce more physically relevant solutions.

With these enhancements, the model we have produced could evolve into a powerful tool for exploring meson spectra, decay processes, and beyond **Standard Model** scenarios involving bound-state dynamics.

References

[A] – Cern Courier https://cerncourier.com/a/a-november-revolution-the-birth-of-a-new-particle/
[B] – Science Direct : https://www.sciencedirect.com/science/article/abs/pii/S0146641016300734/
[C] – IOP Science: https://iopscience.iop.org/book/mono/978-0-7503-3087-9/chapter/bk978-0-75033087-9ch1
[D] – DOAJ : https://doaj.org/article/c41a40c9543741b9b3b1a5acd13e3980
[E] – Arxiv: https://arxiv.org/html/2303.07394v3
[F] – Arxiv: https://arxiv.org/abs/1904.11542

---

[1] Particle Data Group – Quark Masses: https://pdglive.lbl.gov/DataBlock.action?node=Q004M
[2] Van Royen & Weisskopf – Decay Widths: https://doi.org/10.1007/BF02744877
[3] Wikipedia – Quarkonium: https://en.wikipedia.org/wiki/Quarkonium
[4] Eichten et al., Cornell Potential Model: https://doi.org/10.1103/PhysRevD.17.3090
[5] Brambilla et al., Heavy Quarkonium Physics, Rev. Mod. Phys. 77, 1423 (2005): https://arxiv.org/abs/hep-ph/0412158
[6] Wikipedia – Breit Equation: https://en.wikipedia.org/wiki/Breit_equation
[7] Bethe–Salpeter Equation: https://en.wikipedia.org/wiki/Bethe–Salpeter_equation
[8] Hyperfine Structure – Wikipedia: https://en.wikipedia.org/wiki/Hyperfine_structure
[9] PDG – Charmonium States Overview: https://pdglive.lbl.gov/
[10] Numerical Recipes – Shooting Method: https://numerical.recipes/
[11] Arfken & Weber – Mathematical Methods for Physicists
[12] Ar5iv - https://ar5iv.org/html/1805.02534
[13] SciElo Mexico : https://www.scielo.org.mx/scielo.php

'''

The following script is for project title:
"The Potential of charm and Beauty"
By Kevin McCarthy and Daniel Farrell

Solving the radial Schrodinger Equation using a Cornell potential for:
(1) Charmonium: S-waves and P-waves
(2) Bottomonium: S-waves and P-waves

For each of the 4 systems, for lowest 5 eigenenergies: - Plotting graphs of:
(1) Bracket finding
(2) Energy Spectrum
(3) Normalised Wavefunction versus rho and r

- Outputting to console:
(1) Eigenvalues
(2) Wavefunction squared at zero
(3) Mean radius

Glossary of Parts;
(1) PART 1: Constants and 4 Systems Set Up ****** CONTROL LOOP FOR SYSTEMS ******
(2) PART 2: Potential and ODE system
(3) PART 3: Shooting Method
(4) PART 4: Find Brackets and Plot Process
(5) PART 5: Bisection Method
(6) PART 6: Find Eigenvalues and Plot (Dimensionless)
(7) PART 7: Solve and Cache All Wavefunctions Once
(8) PART 8: Solve Schrodinger Equation, Normalise, and Plot (Dimensionless)
(9) PART 9: $|\Psi(0)|^2$ and mean radius
(10) PART 10: Conversion to Standard Units
****** END OF CONTROL LOOP FOR SYSTEMS ******
(11) PART 11: Wavefunction Intensity Heatmap

End of Intro
'''

```
# Importing Libraries

import numpy as np import
matplotlib.pyplot as plt

# ****** PART 1: Constants and 4 Systems Set Up ******

# alpha prime is written as just alpha for brevity, tidyness
alpha = 0.472 # Not actually dimensionless, see dimensionless potential function h = 0.001 # step size for rk4
and rho step in shooting function

rho0 = 1e-5 # initial close to zero rho rhomax = 20 # chosen to include 5
lowest eigenenrgies

hbar_SI = 1.05e-34       # SI units (J·s), used in potential hbar_ev = 6.5821e-16     #
eV·s, used for conversions to meters c = 299792458  # m/s

# Define system combinations
systems = [
```

```
    ('Charmonium', 1.32e9, 0),  # S-wave
    ('Charmonium', 1.32e9, 1),  # P-wave
    ('Bottomonium', 4.9e9, 0),  # S-wave
    ('Bottomonium', 4.9e9, 1)   # P-wave ]

# Dictionary to hold data for all 4 systems all_wavefunction_data = {}


# ****** END OF PART 1: Constants and 4 Systems Set Up ******


# ****** CONTROL LOOP FOR SYSTEMS ******

# Iterates all solving for each of the 4 systems for system_name, m, ell in
systems:     print(f"\n\n Solving for {system_name}, ℓ = {ell} \n")

    # Derived quantities per system
    m2c4 = (m**2)*(c**4)  # For use in kappa, avoids runtime error     kappa = (440**2) / m2c4

    # Reset wavefunction cache per system     wavefunction_data = {}

    # Defining function for initial conditions     def
get_initial_conditions(ell):
        '''
        Returns initial conditions based on angular momentum.
        For S-wave (ell=0): small ψ(0), zero derivative.
        For P-wave (ell=1): zero ψ(0), small derivative.
        '''
        return (1e-8, 0) if ell == 0 else (0, 1e-8)

# ****** ALL FOLLOWING CODE IS WITHIN CONTROL LOOP FOR SYSTEMS ******


    # ****** PART 2: Potential and ODE system ******

    # Defining the Cornell Potential in dimensionless form     def V_rho(rho, alpha,
kappa):
        '''
        Compute the dimensionless Cornell potential at a given rho.

        Parameters         --------
-       rho : float
            Dimensionless radial coordinate.         alpha : float
            Effective coupling constant (dimensionless).
        kappa : float
            Linear confinement coefficient (dimensionless).

        Returns         ------
float
            Value of the potential V(rho).
        '''         if rho == 0:
return 0  # avoid
singularity
        return -(alpha / rho * (hbar_SI**2) * (c**2)) + (kappa * rho)


    # Defining Schrodinger equation in Dimensionless form     def schrodinger(rho,
y, params):
```

```
        '''
        Evaluate the right-hand side of the radial Schrödinger equation.

        Parameters        --------
-        rho : float
            Dimensionless radial coordinate.
        y : ndarray
            Array containing [ψ, ψ'] at current rho.
        params : tuple
            Tuple containing (epsilon, alpha, kappa, ell) for the potential and angular momentum
        Returns        ------
ndarray
            Array of derivatives [dψ/drho, d²ψ/drho²].
        '''        psi, phi = y
        epsilon, alpha, kappa, ell = params        if rho == 0:
            rho = 1e-8  # avoid division by zero
        V = V_rho(rho, alpha, kappa) # tidyness, for use in schrondinger calculation        dpsi_drho = phi
        dphi_drho = (ell*(ell+1)/rho**2 + V - epsilon) * psi        return
np.array([dpsi_drho, dphi_drho])

    # 4th Order Runge-Katta    def rk4_step(f, rho, y,
h, params):
        '''
        Perform a single 4th-order Runge-Kutta integration step.

        Parameters        -------
--        f : function
            The function defining the system of ODEs, must return dy/dx.
        rho : float
            The current value of the independent variable.
        y : ndarray
            Current values of the dependent variables.
        h : float
            Step size for integration.        params : tuple
            Parameters to pass to the ODE function.

        Returns        ------
ndarray
            Updated values of y after one RK4 step.
        '''
        k1 = f(rho, y, params)
        k2 = f(rho + 0.5*h, y + 0.5*h*k1, params)        k3 = f(rho +
0.5*h, y + 0.5*h*k2, params)        k4 = f(rho + h, y + h*k3,
params)        return y + (h/6)*(k1 + 2*k2 + 2*k3 + k4)


    # ****** END OF PART 2: Potential and ODE system ******

    # ****** PART 3: Shooting Method ******
        def shoot(epsilon, alpha, kappa, ell, rho0, rhomax, h):
        '''
        Integrate the Schrödinger equation from rho0 to rhomax using the shooting method.
        Parameters        ---------
epsilon : float
            Trial energy eigenvalue.
        alpha : float
```

```
        Coupling constant for the potential.
    kappa : float
        Confinement term coefficient.
    ell : int
        Angular momentum quantum number.
    rho0 : float
        Starting value for rho (close to zero).
    rhomax : float
        Upper limit of integration.
    h : float           Step size.

    Returns       ------
float
        Value of the wavefunction ψ at rho = rhomax.
    '''
    params = (epsilon, alpha, kappa, ell)       # setting initial
conditions       psi0, dpsi0 = get_initial_conditions(ell)

    y = np.array([psi0, dpsi0])
        rho = rho0       while rho <
rhomax:
        y = rk4_step(schrodinger, rho, y, h, params)           rho += h
        return y[0]  # return ψ at rhomax


# ****** END OF PART 3: Shooting Method ******

# ****** PART 4: Find Brackets and Plot Process ******
    def find_brackets_plot(alpha, kappa, rho0, rhomax, h, eps_min, eps_max, eps_steps):
    '''
    Identify intervals of epsilon with sign changes in ψ(rho_max), indicating roots.
    Parameters       ---------
alpha : float
        Coupling constant for the potential.
    kappa : float
        Confinement term coefficient.
    rho0 : float
        Initial radial value.
    rhomax : float          Final radial
value.
    h : float           Step size.
eps_min : float
        Minimum trial epsilon.
    eps_max : float
        Maximum trial epsilon.
    eps_steps : int
        Number of trial steps between eps_min and eps_max.

    Returns       ------
list of tuples
        Each tuple contains (lower_bound, upper_bound) bracketing an eigenvalue.
    '''
    epsilons = np.linspace(eps_min, eps_max, eps_steps)       values = []
brackets = []

    prev_val = shoot(epsilons[0], alpha, kappa, ell, rho0, rhomax, h)       values.append(prev_val)
```

```
        for eps in epsilons[1:]:            val = shoot(eps, alpha, kappa, ell, rho0, rhomax,
h)            values.append(val)            if prev_val * val < 0:
            brackets.append((eps - (epsilons[1] - epsilons[0]), eps))            prev_val = val

    plt.plot(epsilons, values)
    plt.axhline(0, color='black', linestyle='--')        plt.xlabel('ε')
plt.ylabel('ψ(ρ_max)')
    plt.title('Finding sign changes: ψ(ρ_max) vs ε')        plt.grid(True)
plt.show()

    return brackets


# ****** END OF PART 4: Find Brackets and Plot Process ******

# ****** PART 5: Bisection Method ******
    def bisection(alpha, kappa, ell, rho0, rhomax, h, a, b, tol=1e-6, max_iter=100):
    '''
    Find an eigenvalue using the bisection method within a bracket.

    Parameters        ---------
alpha : float
        Coupling constant for the potential.
    kappa : float
        Confinement term coefficient.
    ell : int
        Angular momentum quantum number.
    rho0 : float
        Starting value of radial coordinate.
    rhomax : float
        End value of radial coordinate.        h : float
        Step size for integration.
a   : float
        Lower bound of epsilon bracket.
b   : float
        Upper bound of epsilon bracket.
    tol : float, optional
        Tolerance for stopping, by default 1e-6.
    max_iter : int, optional
        Maximum number of iterations, by default 100.

    Returns        ------
float
        Estimated eigenvalue ε that satisfies boundary conditions.
    '''
    fa = shoot(a, alpha, kappa, ell, rho0, rhomax, h)        fb = shoot(b, alpha,
kappa, ell, rho0, rhomax, h)
        if fa * fb > 0:
        raise ValueError("No sign change in bracket!")
        for _ in range(max_iter):
        c = (a + b) / 2
        fc = shoot(c, alpha, kappa, ell, rho0, rhomax, h)
            if abs(fc) < tol:
            return c
            if fa * fc < 0:                b, fb
= c, fc            else:
```

```
            a, fa = c, fc

        return (a + b) / 2  # best guess


    # ****** END OF PART 5: Bisection Method ******

    # ****** PART 6: Find Eigenvalues and Plot ******
        brackets = find_brackets_plot(alpha, kappa, rho0, rhomax, h, 0.5, 3.0, 200)

    eigenvalues = []
    for a, b in brackets[:5]:  # first 5 eigenstates
        eigen = bisection(alpha, kappa, ell, rho0, rhomax, h, a, b)         eigenvalues.append(eigen)
        print(f"Eigenvalue found: ε = {eigen:.6f}")
        plt.figure()     for n, ev in enumerate(eigenvalues):
        En = ev*m # mass is already in eV, no need to multiply by c^2
        plt.hlines(En, xmin=0, xmax=1, label=f'n={n}')     plt.title(f'Energy
spectrum: {system_name}, ℓ = {ell}')     plt.xlabel('State index (arbitrary)')
plt.ylabel('ε (dimensionless energy)')     plt.legend()     plt.grid(True)
plt.show()

    # ****** END OF PART 6: Find Eigenvalues and Plot ******

    # ****** PART 7: Solve and Cache All Wavefunctions Once ******

    # This block prevents repeated calls to solve_wavefunction and normalize_wavefunction.
      def solve_wavefunction(epsilon, alpha, kappa, ell, rho0, rhomax, h):
        '''
        Integrate the radial Schrödinger equation to obtain the wavefunction.

        Parameters         ---------         epsilon
: float            Energy eigenvalue.
        alpha : float
            Coupling constant.
        kappa : float
            Confinement strength.
        ell : int
            Angular momentum quantum number.
        rho0 : float
            Initial rho value.
        rhomax : float
            Maximum rho to integrate to.
        h : float            Step size.

        Returns         ------         tuple of ndarrays
rho_values : Array of rho values.
            psi_values : Array of wavefunction values at corresponding rho.
        '''
        params = (epsilon, alpha, kappa, ell)

        # setting initial conditions
        psi0, dpsi0 = get_initial_conditions(ell)

        y = np.array([psi0, dpsi0])
```

```
    rho_values = [rho0]    psi_values = [psi0]
    rho = rho0    while rho <
rhomax:
    y = rk4_step(schrodinger, rho, y, h, params)    rho += h
    rho_values.append(rho)    psi_values.append(y[0])

    return np.array(rho_values), np.array(psi_values)
    def normalize_wavefunction(rho_values, psi_values):
    '''
    Normalize a wavefunction using the spherical volume element.

    Parameters    ---------
rho_values : ndarray
    Array of dimensionless radial coordinates.
    psi_values : ndarray
    Unnormalized wavefunction values.

    Returns    ------
ndarray
    Normalized wavefunction.
    '''
    n = len(rho_values)    integral
= 0    for i in range(n-1):
    drho = rho_values[i+1] - rho_values[i]    integrand_i = rho_values[i]**2 *
psi_values[i]**2    integrand_ip1 = rho_values[i+1]**2 * psi_values[i+1]**2
integral += 0.5 * (integrand_i + integrand_ip1) * drho

    norm_factor = np.sqrt(integral)    normalized_psi = psi_values /
norm_factor
    return normalized_psi

# Precompute and store wavefunctions to avoid recomputation    wavefunction_data = {}
    for i, eigenvalue in enumerate(eigenvalues):
    print(f"Caching wavefunction for state n={i} with ε={eigenvalue:.6f}...")
    rho_vals, psi_vals = solve_wavefunction(eigenvalue, alpha, kappa, ell, rho0, rhomax, h)    norm_psi =
normalize_wavefunction(rho_vals, psi_vals)
    r_vals = rho_vals * (hbar_ev / (m * c))  # Convert to r (meters)
    wavefunction_data[i] = {
"epsilon": eigenvalue,
    "rho": rho_vals,
    "psi_raw": psi_vals,
    "psi_norm": norm_psi,
    "r": r_vals
    }


# ****** END OF PART 7: Solve and Cache All Wavefunctions Once ******
# ****** PART 8: Solve Schrodinger Equation, Normalise, and Plot ******
    plt.figure()    for i in
wavefunction_data:
    rho_values = wavefunction_data[i]["rho"]    normalized_psi =
wavefunction_data[i]["psi_norm"]    plt.plot(rho_values, normalized_psi,
label=f'n={i}')
    plt.xlabel('ρ (dimensionless)')
plt.ylabel('Normalized Ψ(ρ)')
  plt.title(f'Normalized Wavefunctions: {system_name}, ℓ = {ell}')    plt.legend()
plt.grid(True)    plt.show()
```

```
    # ****** END OF PART 8: Solve Schrodinger Equation, Normalise, and Plot ******
    # ****** PART 9: |Ψ(0)|² and mean radius ******

    print("Now Calculating |Ψ(0)|² and r₀ (mean radius) for each n")

    for i in wavefunction_data:        eigenvalue =
wavefunction_data[i]["epsilon"]        rho_values =
wavefunction_data[i]["rho"]        normalized_psi =
wavefunction_data[i]["psi_norm"]        r_values = wavefunction_data[i]["r"]
        print(f"\nState n={i} with ε={eigenvalue:.6f}")

        # 1. |Ψ(0)|² in ρ and r        psi0_rho =
normalized_psi[0]        psi0_squared_rho =
abs(psi0_rho)**2
        psi0_squared_r = psi0_squared_rho / (hbar_ev / (m * c))

        print(f"|Ψ(0)|² (in ρ): {psi0_squared_rho:.5e}")        print(f"|Ψ(0)|² (in r):
{psi0_squared_r:.5e} m⁻³")

        # 2. Mean radius via expectation value
        mean_r_rho = np.trapz(rho_values**3 * normalized_psi**2, rho_values)        mean_r =
mean_r_rho * (hbar_ev / (m * c))        print(f"⟨r⟩ (expectation value): {mean_r:.5e} m")

        # 3. Width at half height
        half_max = max(abs(normalized_psi)) / 2
        indices = np.where(abs(normalized_psi) >= half_max)[0]
            if len(indices) >= 2:
            width_half_height_rho = rho_values[indices[-1]] - rho_values[indices[0]]            width_half_height_r =
width_half_height_rho * (hbar_ev / (m * c))            print(f"r₀ (width at half max): {width_half_height_r:.5e} m")
else:
            print("Could not determine r₀ (half-width)")

    # ****** END OF PART 9: |Ψ(0)|² and mean radius ******
    # ****** PART 10: Conversion to Standard Units ******

    print("\n Wavefunctions and Energies in Dimensional r ")

    plt.figure()
        for i in wavefunction_data:
        epsilon = wavefunction_data[i]["epsilon"]        r_values =
wavefunction_data[i]["r"]
        normalized_psi = wavefunction_data[i]["psi_norm"]
            energy_ev = epsilon * m # Again, mass is already in eV, no need to multiply by c^2            energy_mev = energy_ev /
1e6
            plt.plot(r_values * 1e15, normalized_psi, label=f'n={i} ({energy_mev:.1f} MeV)')            print(f"State n={i}: E =
{energy_mev:.2f} MeV")

    plt.xlabel('r (fm)')    plt.ylabel('Normalized Ψ(r)')
    plt.title(f'Wavefunctions in Dimensional r: {system_name}, ℓ = {ell}')    plt.legend()
plt.grid(True)    plt.show()

    # ****** END OF PART 10: Conversion to Standard Units ******

    # Store data for combined heatmap plotting
    key = f"{system_name}, ℓ={ell}"
    all_wavefunction_data[key] = wavefunction_data
```

```
# ****** END OF CONTROL LOOP FOR SYSTEMS ******

# ****** PART 11: Heatmap Graph ******

# Plot a 2×2 grid of wavefunction heatmaps for all systems def
plot_all_wavefunction_heatmaps(data_dict):
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))     axes = axes.flatten()

    for ax, (label, wavefunction_data) in zip(axes, data_dict.items()):
        # Interpolate wavefunctions onto a common r grid          r_values =
wavefunction_data[0]["r"]
        grid_r = np.linspace(r_values.min(), r_values.max(), 400)          heatmap = []

        for n in range(5):
            psi = wavefunction_data[n]["psi_norm"]             r_n =
wavefunction_data[n]["r"]             psi_interp = np.interp(grid_r, r_n,
psi)             heatmap.append(psi_interp)          heatmap =
np.array(heatmap)

        im = ax.imshow(heatmap, extent=[grid_r[0]*1e15, grid_r[-1]*1e15, 4, 0],                       aspect='auto',
cmap='viridis')          ax.set_title(label, fontsize=10)          ax.set_xlabel('r (fm)', fontsize=9)
ax.set_ylabel('State index n', fontsize=9)          ax.tick_params(labelsize=8)

    # Add shared colorbar     fig.subplots_adjust(right=0.88)
    cbar_ax = fig.add_axes([0.90, 0.15, 0.02, 0.7])     fig.colorbar(im, cax=cbar_ax,
label='Ψ(r)')

    plt.suptitle("Wavefunction Heatmaps for Charmonium & Bottomonium (S and P waves)", fontsize=     plt.tight_layout(rect=[0, 0,
0.88, 0.95])     plt.show()

# Call the function to draw the combined heatmap plot_all_wavefunction_heatmaps(all_wavefunction_data)

# ****** END OF PART 11: Heatwave Graph ******

'''
END OF SCRIPT
'''
```

```
'''
This script compares computed energy levels with experimental data from the PDG. For each of the four
systems (charmonium and bottomonium, S- and P-waves), it:

(1)  Plots the computed vs PDG energy levels.
(2)  Calculates and visualizes differences in level spacings (residuals).

Each comparison is shown in a two-part figure:
-  Top subplot: energy levels
-  Bottom subplot: spacing residuals (ΔE_computed - ΔE_PDG)

Missing data points (None) are automatically filtered.
'''

import matplotlib.pyplot as plt import numpy as
np


# ****** FUNCTION: PLOT ENERGY COMPARISON ******

def plot_energy_comparison(n_values, computed, pdg, label):
    '''
    Plots a 2-panel comparison between computed and PDG energy levels.

    Parameters     ---------     n_values
: list of int
        Indices for the energy levels (e.g., 0 to 4).
    computed : list of float         Computed energy
levels (MeV).
    pdg : list of float
        Experimental PDG energy levels (MeV).
    label : str
        Label describing the system (e.g., "Charmonium ℓ = 0").

    Returns
    -------
    None. Displays the plot.
    '''

    # Set up a 2-row figure: top = energies, bottom = spacing residuals      fig, axs = plt.subplots(2, 1,
figsize=(8, 6), sharex=True,                          gridspec_kw={'height_ratios': [3, 1]})

    # ****** PLOT 1: ENERGY LEVELS ******
    axs[0].plot(n_values, computed, 'o-', label='Computed', color='blue')     axs[0].plot(n_values, pdg,
'o--', label='PDG', color='orange')     axs[0].set_ylabel("Energy (MeV)")     axs[0].set_title(f"{label}
— Energy Levels")     axs[0].legend()     axs[0].grid(True)

    # ****** PLOT 2: RESIDUALS OF ENERGY SPACING ******
spacing_computed = np.diff(computed)     spacing_pdg = np.diff(pdg)

    # Residuals: difference in spacings (ΔE_computed - ΔE_PDG)      spacing_diff =
spacing_computed - spacing_pdg
    n_spacing = [f"{i}-{i+1}" for i in range(len(spacing_diff))]     axs[1].bar(n_spacing, spacing_diff,

color='purple')     axs[1].axhline(0, color='black', linestyle='--')     axs[1].set_ylabel("ΔΔE (MeV)")

axs[1].set_xlabel("State spacing")
```

```
    axs[1].set_title("Residuals: Computed ΔE - PDG ΔE")     axs[1].grid(True)

    plt.tight_layout()     plt.show()


# ****** SYSTEM DATA ******

# Experimental and computed energy levels (in MeV)

# Charmonium ℓ = 0 (S-wave)
pdg_charmonium_s = [3097, 3686, 3773, 4039, 4191]
computed_charmonium_s = [983.37, 1381.36, 1829.10, 2359.75, 2940.15]

# Charmonium ℓ = 1 (P-wave)
pdg_charmonium_p = [3415, 3511, 3556, 3525, None]  # Final level missing
computed_charmonium_p = [979.22, 1368.92, 1824.95, 2343.17, 2936.01]

# Bottomonium ℓ = 0 (S-wave)
pdg_bottomonium_s = [9399, 9999, 10355, 10579, 10860]
computed_bottomonium_s = [3650.38, 5127.76, 6789.82, 8759.67, 10914.20]

# Bottomonium ℓ = 1 (P-wave)
pdg_bottomonium_p = [9859, 9893, 9912, 9899, None]  # Final level missing
computed_bottomonium_p = [3634.99, 5081.60, 6774.43, 8698.12, 10898.81]



# ****** DATA CLEANING FUNCTION ******

def clean_data(computed, pdg):
    '''
    Removes any levels where either computed or PDG value is missing (None).

    Parameters     ---------
computed : list
        Computed energy values.
    pdg : list
        Experimental PDG values.

    Returns     ------     tuple:         cleaned_computed : list of valid computed
values         cleaned_pdg : list of corresponding PDG values         indices : list of
int indices (for x-axis plotting)
    '''
    cleaned_c, cleaned_p = [], []     for c, p in zip(computed,
pdg):         if c is not None and p is not None:
            cleaned_c.append(c)             cleaned_p.append(p)
    return cleaned_c, cleaned_p, list(range(len(cleaned_c)))
# ****** PLOT ALL FOUR SYSTEMS ******

# Charmonium ℓ = 0
c_c, p_c, n_vals = clean_data(computed_charmonium_s, pdg_charmonium_s)
plot_energy_comparison(n_vals, c_c, p_c, "Charmonium ℓ = 0")

# Charmonium ℓ = 1
c_c, p_c, n_vals = clean_data(computed_charmonium_p, pdg_charmonium_p)
plot_energy_comparison(n_vals, c_c, p_c, "Charmonium ℓ = 1")

# Bottomonium ℓ = 0
```

```
c_c, p_c, n_vals = clean_data(computed_bottomonium_s, pdg_bottomonium_s)
plot_energy_comparison(n_vals, c_c, p_c, "Bottomonium ℓ = 0")

# Bottomonium ℓ = 1
c_c, p_c, n_vals = clean_data(computed_bottomonium_p, pdg_bottomonium_p)
plot_energy_comparison(n_vals, c_c, p_c, "Bottomonium ℓ = 1")
```

```
'''
This script compares numerical results obtained from solving the radial Schrodinger equation,
(for charmonium and bottomonium systems) in Charm_and_Beauty.py with
experimental values from the Particle Data Group (PDG).
The computed and  experimental PDG values are displayed in side-by-side tables, including their absolute
differences.

Results compared:
-   Energy levels
-   Wavefunction at origin: |Ψ(0)|²
-   Mean radius <r>

Tables are displayed using matplotlib, and data is structured with pandas.
'''

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd  # Used for tabular data manipulation and formatting


def show_table_with_diff(computed, reference, headers, title, unit):
    '''
    Display a comparison table between computed and reference values.

    Parameters     -
---------
    computed : list of float
        Computed values (from numerical simulation).
    reference : list of float or None
        Experimental reference values (from PDG). Use None where unavailable.
    headers : list of str
        List containing the label for the computed column.
    title : str
        Title displayed above the table (e.g., "Charmonium Energies").
    unit : str
        Unit to be displayed (e.g., "MeV", "m", "m-³").

    Returns
    -------
    None. Displays a styled matplotlib table comparing values.

'''
ro
w
s
=
[]

    for c, r in zip(computed, reference):         # Handle missing experimental
data         if r is None:             diff = "n/a"             r_val = "n/a"         else:
diff = f"{abs(c - r):.4g}  {unit}"  # Absolute difference             r_val = r
        rows.append([c, r_val, diff])

    # Create DataFrame for table formatting and potential manipulation
    col_labels = [headers[0], f"Experimental PDG ({unit})", f"Difference ({unit})"]     df =
pd.DataFrame(rows, columns=col_labels).replace({np.nan: "n/a"})

    # Create and configure matplotlib figure
```

```
    fig, ax = plt.subplots(figsize=(9, len(df) * 0.6 + 0.8), dpi=150)     ax.axis('off')  # Hide the axes
frame
    # Add the data table
table = ax.table(
cellText=df.values,
colLabels=df.columns,
cellLoc='center',
loc='center',
        colColours=["#dddddd"] * len(df.columns),  # Light gray header
    )
    table.scale(1.2, 1.3)  # Scale to improve legibility

    # Add title to the figure     plt.subplots_adjust(top=0.85)
    plt.title(title, fontsize=13, weight='bold', pad=1)     plt.show()


# ****** Experimental PDG values (source in report) ******
pdg_charmonium_s = [3097, 3686, 3773, 4039, 4191]
pdg_charmonium_p = [3415, 3511, 3556, 3525, None]
pdg_bottomonium_s = [9399, 9999, None, None, None]
pdg_bottomonium_p = [9859, 9893, 9912, 9899, None]

pdg_psi0_charmonium = [0.081, None, None, None, None] pdg_psi0_bottomonium
= [0.512, None, None, None, None]

pdg_r_charmonium = [0.25e-15, 0.45e-15, 0.65e-15, None, None] pdg_r_bottomonium =
[0.14e-15, 0.28e-15, 0.42e-15, None, None]


# ****** Computed values from Charm_and_Beauty.py ******

# All energies are in MeV, wavefunction density in m-³m-³, and radii in meters
computed_data = {     "charmonium_s": {
        "energies": [983.37, 1381.36, 1829.10, 2359.75, 2940.15],
        "psi0": [2.83e48] * 5,
        "r_mean": [3.96e-51] * 5
    },
    "charmonium_p": {
        "energies": [979.22, 1368.92, 1824.95, 2343.17, 2936.01],
        "psi0": [0] * 5,  # â=1 â Ï(0) = 0 by angular momentum
        "r_mean": [3.96e-51] * 5
    },
    "bottomonium_s": {
        "energies": [3650.38, 5127.76, 6789.82, 8759.67, 10914.20],
        "psi0": [1.05e49] * 5,
        "r_mean": [1.07e-51] * 5
    },
    "bottomonium_p": {
        "energies": [3634.99, 5081.60, 6774.43, 8698.12, 10898.81],
        "psi0": [0] * 5,
        "r_mean": [1.07e-51] * 5
    }
}


# ****** Display Comparison Tables ******

# (1) Energies (S-wave and P-wave)
```

```
show_table_with_diff(computed_data["charmonium_s"]["energies"], pdg_charmonium_s,
["Computed (MeV)"], "Charmonium Energies (l = 0)", "MeV")
show_table_with_diff(computed_data["charmonium_p"]["energies"], pdg_charmonium_p,
["Computed (MeV)"], "Charmonium Energies (l = 1)", "MeV")

show_table_with_diff(computed_data["bottomonium_s"]["energies"], pdg_bottomonium_s,
["Computed (MeV)"], "Bottomonium Energies (l = 0)", "MeV")

show_table_with_diff(computed_data["bottomonium_p"]["energies"], pdg_bottomonium_p,
["Computed (MeV)"], "Bottomonium Energies (l = 1)", "MeV")

# (2) |Ψ(0)|² values
show_table_with_diff(computed_data["charmonium_s"]["psi0"], pdg_psi0_charmonium,
["|Ψ(0)|² (m-³)"], "|Ψ(0)|² - Charmonium l = 0", "m-³")

show_table_with_diff(computed_data["charmonium_p"]["psi0"], [0]*5,
                     ["|Ψ(0)|² (m-³)"], "|Ψ(0)|² - Charmonium l = 1", "m-³")

show_table_with_diff(computed_data["bottomonium_s"]["psi0"], pdg_psi0_bottomonium,
["|Ψ(0)|² (m-³)"], "|Ψ(0)|² - Bottomonium l = 0", "m-³")

show_table_with_diff(computed_data["bottomonium_p"]["psi0"], [0]*5,
                     ["|Ψ(0)|² (m-3)"], "|Ψ(0)|² - Bottomonium l = 1", "m-³")

# (3) Mean radius <r>
show_table_with_diff(computed_data["charmonium_s"]["r_mean"], pdg_r_charmonium,
["<r>(m)"], "Mean Radius <r> - Charmonium l = 0", "m")

show_table_with_diff(computed_data["bottomonium_s"]["r_mean"], pdg_r_bottomonium,
                     ["<r>(m)"], "Mean Radius <r> - Bottomonium l = 0", "m")
```